

Healthcare Context Intelligence Engine (HCIE) v0.3

- [Healthcare Context Intelligence Engine \(HCIE\) — v0.3](#)

Healthcare Context Intelligence Engine (HCIE) — v0.3

Unified Architecture Blueprint

Date: July 17, 2026 **Project Type:** Open-source, MCP-native orchestration platform **Positioning:** A general-purpose Context Intelligence Engine, specialized for healthcare resource orchestration, using text-only input **Primary Language:** Python 3.11+

***Final Vision:** A Context Intelligence Engine that orchestrates healthcare resources using MCP, hybrid retrieval, context graphs, and agentic planning — based on text input only.*

Table of Contents

1. Project Overview & Positioning
 2. First Principle & Scope
 3. Version Definition (v0.3)
 4. Core Workflow
 5. Architecture Overview
 6. Engine Layers (7 layers)
 7. MCP Gateway
 8. Agent Architecture
 9. Repository Structure
 10. File-by-File Responsibilities
 11. Data Models and Interfaces
 12. Core Algorithms
 13. Sequence Flows
 14. MCP Tool Specifications
 15. Evaluation Framework
 16. Roadmap
 17. Research Differentiation
 18. Open-Source Strategy
-

1. Project Overview & Positioning

The **Healthcare Context Intelligence Engine (HCIE)** merges two prior blueprints into a single coherent system:

- The **general Context Intelligence Engine (CIE)**, which treats context construction — not raw retrieval, not generation — as the first-class problem to solve before an LLM acts.
- The **Healthcare Context Orchestrator v0.2**, which narrowed that idea to a buildable, hackathon-scale, text-only healthcare emergency-orchestration system.

The unifying insight is that both systems solve the *same abstract problem*: given a noisy, underspecified request (a coding task, or a medical emergency description) and a large ecosystem of heterogeneous knowledge sources (a codebase, or patient/hospital/doctor records), **construct the best possible structured context, reason over it, and decide what should happen next** — before invoking any generative or execution step.

HCIE keeps this framing explicit:

- It is **not** a diagnostic system, an image classifier, or a hospital management platform.
- It **is** a context-assembly and orchestration engine that happens to operate in the healthcare emergency domain, built on the exact same seven-layer engine, MCP gateway, hybrid retrieval stack, and context-graph reasoning that the general CIE defines for software engineering.

This dual positioning is a deliberate strength: the retrieval/ranking/graph/orchestration core is **domain-agnostic**, and healthcare is the first (and most narratively compelling) vertical demonstration of it. A judge, reviewer, or contributor can see the same architecture apply to “help me write a JWT auth module” and “help me route a chest-pain patient to the nearest cardiac-capable hospital.”

Key positioning statements

- HCIE sits between a *request* (text) and a *knowledge ecosystem* (patient history, hospitals, doctors, emergency contacts, maps).
- It transforms retrieval into reasoning via a **Context Graph**, not just a ranked list of chunks.
- It produces an **execution plan**, not a diagnosis — the clinical judgment always remains with clinicians and emergency responders.
- It is MCP-native: every knowledge source and every action is exposed as an MCP tool, so any MCP-capable client (Claude, ChatGPT, a hospital ops console) can drive it.

2. First Principle & Scope

First Principle: *Build context, not diagnosis. Orchestrate action, not medicine.*

The system answers exactly one question:

“Given the current context, what should happen next?”

In scope (v0.3):

- Text-only input: emergency type, optional patient name, free-text symptoms, location.
- Structured intent extraction from that text.
- Retrieval and ranking over mock/structured healthcare data (patients, hospitals, doctors, emergency contacts).
- A context graph that links patient → symptoms → history → hospital → doctor → emergency contact.
- An agentic planning pipeline that proposes a ranked, explainable next-action plan.
- Human-in-the-loop approval before any “execution” tool (call ambulance, notify family) actually fires.

Explicitly out of scope (carried over from the Healthcare Context Orchestrator’s scope reduction):

- Face recognition, photo/video/ECG/blood-test analysis, wearables, IoT sensors.
- Disease diagnosis or treatment recommendation of any kind.
- Real integrations with insurance, medicine delivery, or ambulance marketplaces.
- Any claim of clinical accuracy — all hospital/doctor/patient data in v0.3 is mock, JSON-backed data.

This scope keeps the system legally, ethically, and technically buildable by a solo developer or small team within a hackathon or early open-source timeline, while still fully exercising every layer of the general CIE architecture.

3. Version Definition (v0.3)

v0.3 product definition: HCIE v0.3 takes the six-agent, four-MCP-server, six-node-graph healthcare orchestrator from v0.2 and re-implements it *inside* the full seven-layer CIE engine, with the CIE’s gateway, discovery-agent pattern, context-graph/pattern-inference machinery, prompt/plan builder, and validation layer all present — just re-targeted from “codebase + GitHub” to “patient + hospitals + doctors + emergency contacts + maps.”

Input form (unchanged from v0.2, kept intentionally minimal):

```
Emergency Type
Patient Name (optional)
Symptoms (free text)
Location
```

Example input:

```
Patient: John
Emergency Type: Medical
Symptoms:
- Chest pain
- Difficulty breathing
- Sweating
- Unconscious
Location: Kakkanad
```

Example output (the planned execution, not a diagnosis):

```
1. Call ambulance.
2. Notify family.
3. Navigate to Hospital A.
4. Share patient summary.
5. Connect to available physician.
```

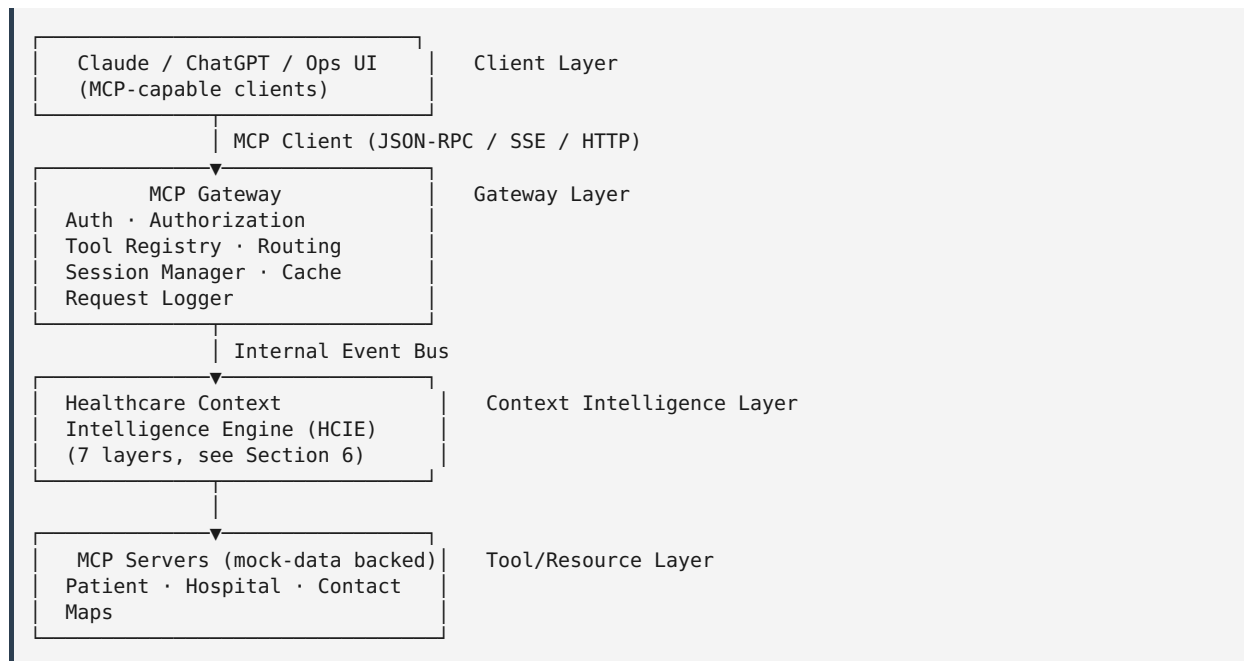
4. Core Workflow

```
User
↓
Enter Emergency Text (type, name, symptoms, location)
↓
Intent Understanding          (Layer 1)
↓
Planning                      (Layer 2)
↓
Knowledge Discovery           (Layer 3) – patient / hospitals / doctors / contacts / maps
↓
Hybrid Retrieval & Rerank     (Layer 4)
↓
Context Graph Construction (Layer 5)
↓
Pattern / Priority Inference (Layer 5)
↓
Plan (Prompt) Building        (Layer 6)
↓
LLM Generation of Ranked Plan (Layer 6)
↓
Validation of Plan            (Layer 7)
↓
Human Approval Gate
↓
Execution (MCP tool calls: ambulance, notify, navigate, share, connect)
```

This is the same skeleton as the CIE’s “User Request → ... → Response” sequence flow (Section 13.1 of the CIE blueprint), with healthcare-specific nodes substituted at each stage.

5. Architecture Overview

Three coordination zones, identical in shape to the general CIE:



The gateway is the single stable endpoint any MCP client talks to; the engine is the reasoning core; the servers are thin, swappable, JSON-mock-backed tool surfaces (future v2 target: real hospital/EHR integrations behind the same interface).

6. Engine Layers

All seven CIE layers are preserved. Each is re-scoped to the healthcare domain.

6.1 Intent Understanding

Purpose: Convert an unstructured emergency report into structured, machine-operable intent.

Files: `intent_classifier.py`, `intent_parser.py`, `goal_extractor.py`, `emergency_type_taxonomy.py` (4 files)

Example

Input:

```
Patient: John
Symptoms: Chest pain, difficulty breathing, sweating, unconscious
Location: Kakkanad
```

Output:

```
{
  "task": "emergency_orchestration",
  "emergency_type": "cardiac_suspected",
  "patient_name": "John",
  "symptoms": ["chest_pain", "difficulty_breathing", "sweating", "unconscious"],
  "location": "Kakkanad",
  "severity_hint": "high"
}
```

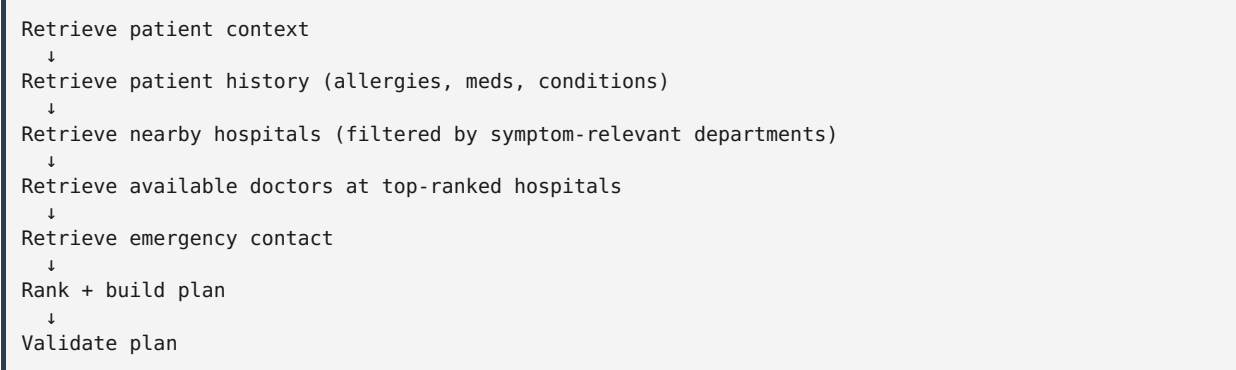
`severity_hint` is a coarse, rules-based urgency signal (not a diagnosis) used only to prioritize retrieval and planning order downstream.

6.2 Planning

Purpose: Build the execution graph — what to retrieve, in what order, and why — mirroring the CIE’s `planner.py` / `task_graph.py` pattern.

Files: `planner.py`, `task_graph.py`, `execution_plan.py` (3 files)

Example planning chain



6.3 Knowledge Discovery

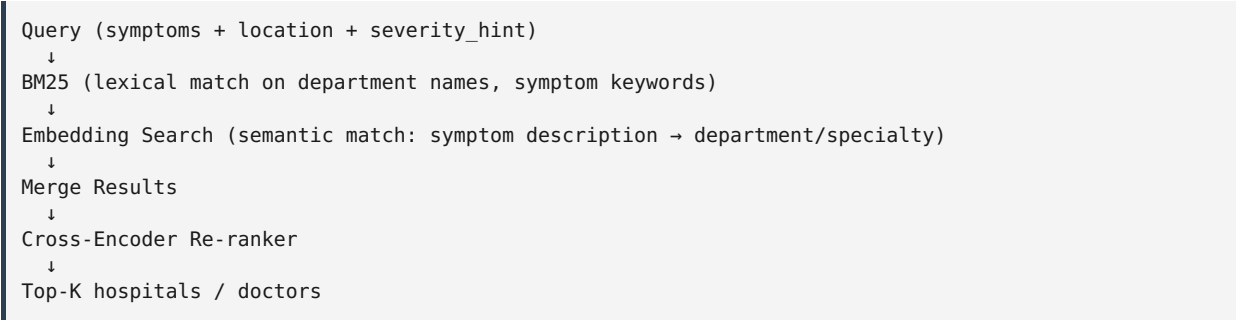
Purpose: Discover context through specialized, single-responsibility discovery agents — same pattern as CIE’s `filesystem_agent.py` / `github_agent.py`, but pointed at healthcare MCP servers.

Files: `patient_discovery_agent.py`, `hospital_discovery_agent.py`, `doctor_discovery_agent.py`, `contact_discovery_agent.py`, `maps_discovery_agent.py` (5 files)

Each agent does exactly one job: call one MCP server, normalize its response into the shared retrieval-result schema (Section 11), and hand off.

6.4 Retrieval and Re-ranking

Purpose: Identical hybrid-retrieval pipeline to the general CIE, applied to hospital/doctor records instead of code files.



Files: `hybrid_retriever.py`, `reranker.py`, `bm25.py`, `embedding_search.py`, `merge.py` (5 files)

Recommended models (unchanged from CIE): `BAAI/bge-m3` or `nomic-embed-text` for embeddings; `BAAI/bge-reranker-v2-m3` or `jina-reranker-v2` for cross-encoder reranking. Lexical search recovers exact matches like “Cardiology” or “ICU”; embedding search recovers semantically related but differently-worded matches (e.g., “chest tightness” → Cardiology).

6.5 Context Intelligence

Purpose: The strongest original contribution, carried over directly: build a **Context Graph** instead of injecting raw retrieved chunks, then reason over it.

Node types (healthcare-specialized, mapped 1:1 onto the CIE’s generic node/edge concept):

CIE generic node	Healthcare node
Repository	Patient

Module/Class	Symptom / History item
Function	Emergency Contact
API	Hospital
Design Pattern	Doctor / Department
README	Case Summary

Edge types: `has_symptom`, `has_history`, `nearest_to`, `staffed_by`, `notifies`, `routed_to`

Files: `context_graph.py`, `pattern_inference.py` (renamed `priority_inference.py` in healthcare context) (2 files)

Reasoning over the graph infers things like: *“Patient has known heart disease + reports chest pain → prioritize hospitals with Cardiology + ICU + ambulance_available=true, within shortest distance.”* This is the healthcare analog of the CIE’s architectural-pattern inference (e.g., detecting “JWT middleware” usage) — same graph-reasoning mechanism, different node semantics.

6.6 Generation

Purpose: Convert the structured context graph + priority inference into an optimized generation request — here, a **ranked action plan** rather than code.

```

Context Graph + Priority Inference
↓
Plan Compressor (keep only top-ranked hospital/doctor/contact)
↓
Plan Prompt Builder
↓
LLM
↓
Ranked Execution Plan (text)

```

Files: `plan_builder.py`, `context_compressor.py`, `plan_optimizer.py` (3 files)

6.7 Validation

Purpose: Ensure the generated plan is safe, complete, and non-diagnostic before it reaches the human approval gate.

```

Schema Check (does plan reference only real, retrieved entities?)
↓
Safety Check (no diagnostic language, no medication dosing advice)
↓
Completeness Check (ambulance + hospital + contact + doctor all addressed)
↓
Explain Decisions (why this hospital, why this doctor)

```

Files: `plan_validator.py`, `safety_checker.py`, `explanation_generator.py` (3 files)

This closes the loop, mirroring the CIE’s compile/lint/static-analysis/test-generation validation stage — except “correctness” here means “plan references verified context and contains no medical judgment,” not “code compiles.”

Layer file count total: $4 + 3 + 5 + 5 + 2 + 3 + 3 = 25$ files

7. MCP Gateway

The gateway is unchanged in role from the CIE: one stable endpoint for capability discovery, tool access, session continuity, auth, and logging — now fronting healthcare MCP servers instead of code-discovery servers.

Proposed structure

```
gateway/
├── gateway.py
├── router.py
├── auth.py
├── session.py
├── registry.py
├── cache.py
├── logger.py
├── tool_registry.py
└── capability_registry.py
```

(9 files — identical structure to the general CIE gateway)

Responsibilities

- Register MCP tools from all four healthcare servers (Patient, Hospital, Emergency Contact, Maps).
- Discover capabilities per session (an MCP client asks “what can you do?”).
- Route requests to the correct discovery agent / MCP server.
- Cache hospital/doctor lookups within a session (avoid re-querying static mock data).
- Track session state (a single emergency case as it moves through planning → approval → execution).
- Enforce an **approval gate**: execution-class tools (`call_ambulance` , `notify_family`) are registered but gateway-blocked until an explicit human approval event is received.
- Log every tool call for auditability — important in a healthcare-adjacent context even with mock data, to establish the pattern early.

8. Agent Architecture

Merges the CIE’s broader multi-agent tree with the v0.2 orchestrator’s leaner six-agent set. The Chief Agent stays at the top; the six original orchestrator agents become the “spine,” and CIE-style specialized sub-agents are added underneath **Retrieval** and **Context** stages to preserve the general engine’s granularity.

```
Chief Agent
├── Intent Agent
├── Context Agent
│   ├── Patient Discovery Agent
│   ├── History Discovery Agent
│   └── Emergency Contact Discovery Agent
├── Planning Agent
├── Retrieval Agent
│   ├── Hospital Discovery Agent
│   ├── Doctor Discovery Agent
│   ├── Maps Discovery Agent
│   └── Ranking Agent
├── Context Builder Agent      (builds the graph)
├── Plan Builder Agent        (was "Prompt Builder Agent" in CIE)
├── Validation Agent
└── Execution Agent          (approval-gated MCP tool calls)
```

Agent	Responsibility
Chief Agent	Coordinates the end-to-end workflow, enforces guardrails and the approval gate.
Intent Agent	Parses the emergency text into structured intent.
Context Agent	Gathers patient identity, symptoms, and history via its discovery sub-agents.
Planning Agent	Builds the retrieval/execution task graph.
Retrieval Agent	Runs hospital/doctor/maps discovery, hybrid retrieval,

	and reranking.
Context Builder Agent	Assembles the Context Graph and runs priority inference.
Plan Builder Agent	Compresses context and builds the LLM plan-generation prompt.
Validation Agent	Runs schema, safety, and completeness checks on the generated plan.
Execution Agent	Calls MCP tools only after explicit human approval.

This keeps the v0.2 orchestrator’s disciplined, small “spine” (Chief → Intent → Context → Planning → Retrieval → Execution) while restoring the CIE’s finer-grained specialized sub-agents where they add real value (hospital vs. doctor vs. maps discovery are genuinely different data sources, just as GitHub vs. filesystem vs. docs discovery were in the original CIE).

9. Repository Structure

```

healthcare-context-intelligence-engine/
├── apps/
│   ├── gateway/                (re-exports gateway/ below)
│   └── api/                    (REST/HTTP entrypoint for the demo UI)
│       ├── main.py
│       └── schemas.py
├── gateway/                    [9 files]
│   ├── gateway.py
│   ├── router.py
│   ├── auth.py
│   ├── session.py
│   ├── registry.py
│   ├── cache.py
│   ├── logger.py
│   ├── tool_registry.py
│   └── capability_registry.py
├── engine/
│   ├── intent/                 [4 files]
│   │   ├── intent_classifier.py
│   │   ├── intent_parser.py
│   │   ├── goal_extractor.py
│   │   └── emergency_type_taxonomy.py
│   ├── planning/              [3 files]
│   │   ├── planner.py
│   │   ├── task_graph.py
│   │   └── execution_plan.py
│   ├── discovery/             [5 files]
│   │   ├── patient_discovery_agent.py
│   │   ├── hospital_discovery_agent.py
│   │   ├── doctor_discovery_agent.py
│   │   ├── contact_discovery_agent.py
│   │   └── maps_discovery_agent.py
│   ├── retrieval/             [5 files]
│   │   ├── hybrid_retriever.py
│   │   ├── reranker.py
│   │   ├── bm25.py
│   │   ├── embedding_search.py
│   │   └── merge.py
│   ├── context/               [2 files]
│   │   ├── context_graph.py
│   │   └── priority_inference.py
│   ├── generation/           [3 files]
│   │   ├── plan_builder.py
│   │   ├── context_compressor.py
│   │   └── plan_optimizer.py
│   └── validation/            [3 files]
│       ├── plan_validator.py
│       ├── safety_checker.py
│       └── explanation_generator.py
└── agents/
    ├── chief/chief_agent.py
    ├── intent/intent_agent.py
    └── context/context_agent.py

```



```
| | | | | planning/planning_agent.py
| | | | | retrieval/retrieval_agent.py
| | | | | context_builder/context_builder_agent.py
| | | | | plan_builder/plan_builder_agent.py
| | | | | validation/validation_agent.py
| | | | | execution/execution_agent.py [9 files]
| | servers/ [4 files]
| | | | | patient_context_server.py
| | | | | hospital_server.py
| | | | | emergency_contact_server.py
| | | | | maps_server.py
| | mock_data/ [4 files]
| | | | | patients.json
| | | | | hospitals.json
| | | | | doctors.json
| | | | | emergency_contacts.json
| | ranking/ [1 file]
| | | | | context_intelligence_score.py
| | graph/ [1 file]
| | | | | graph_schema.py
| | schemas/ [5 files]
| | | | | intent.py
| | | | | retrieval_result.py
| | | | | graph_node.py
| | | | | plan.py
| | | | | mcp_payloads.py
| | connectors/
| | | | | maps/haversine_distance.py [1 file]
| | tests/
| | | | | unit/ [~10 files]
| | | | | integration/ [~5 files]
| | | | | evaluation/ [~5 files]
| | docs/
| | | | | prd/
| | | | | architecture/
| | | | | research/
| | | | | diagrams/
| | examples/ [3 files]
| | | | | cardiac_emergency.json
| | | | | trauma_emergency.json
| | | | | pediatric_emergency.json
| | scripts/ [2 files]
| | | | | seed_mock_data.py
| | | | | run_demo.py
| | configs/ [2 files]
| | | | | settings.yaml
| | | | | mcp_servers.yaml
| | README.md
| | LICENSE
```

Estimated total file count for MVP: ~85-95 files (Python + JSON + config + docs), which is realistic for a solo builder or small team within a 4-8 week open-source push, or a scoped-down ~35-file slice for a hackathon weekend (gateway + one discovery agent per server + hybrid retrieval + context graph + plan builder + validation, skipping the full test suite and docs tree).

10. File-by-File Responsibilities

Area	File	Responsibility
Intent	intent_classifier.py	Classifies emergency type (cardiac, trauma, respiratory, etc.).
Intent	intent_parser.py	Extracts structured fields: patient name, symptoms, location.
Intent	goal_extractor.py	Detects implicit urgency/severity hints from phrasing.
Intent	emergency_type_taxonomy.py	Static taxonomy mapping symptom clusters to emergency categories.

Planning	<code>planner.py</code>	Builds the retrieval/execution plan order.
Planning	<code>task_graph.py</code>	Represents the case as a dependency graph of retrieval steps.
Planning	<code>execution_plan.py</code>	Serializes the plan into executable MCP tool calls.
Discovery	<code>patient_discovery_agent.py</code>	Calls Patient Context Server for identity/history.
Discovery	<code>hospital_discovery_agent.py</code>	Calls Hospital Server for nearby/relevant hospitals.
Discovery	<code>doctor_discovery_agent.py</code>	Calls Hospital Server for available doctors.
Discovery	<code>contact_discovery_agent.py</code>	Calls Emergency Contact Server.
Discovery	<code>maps_discovery_agent.py</code>	Calls Maps Server for distance/route/current location.
Retrieval	<code>hybrid_retriever.py</code>	Runs BM25 + embedding retrieval over hospital/doctor records.
Retrieval	<code>reranker.py</code>	Cross-encoder reranking of candidates against the query.
Context	<code>context_graph.py</code>	Builds nodes/edges: patient, symptoms, history, hospital, doctor, contact.
Context	<code>priority_inference.py</code>	Infers priority resources from recurring graph signals (e.g., cardiac history + chest pain → Cardiology-first).
Generation	<code>plan_builder.py</code>	Produces the LLM-ready plan-generation prompt.
Generation	<code>context_compressor.py</code>	Compresses graph + retrieval results within token budget.
Validation	<code>plan_validator.py</code>	Confirms plan references only verified, retrieved entities.
Validation	<code>safety_checker.py</code>	Rejects any diagnostic or dosing language in the generated plan.
Validation	<code>explanation_generator.py</code>	Produces the human-readable “why this hospital/doctor” rationale.
Ranking	<code>context_intelligence_score.py</code>	Combines relevance + healthcare-specific quality signals (see Section 12.1).
Servers	<code>patient_context_server.py</code>	MCP server exposing <code>get_patient_summary</code> , <code>get_allergies</code> , <code>get_medications</code> .
Servers	<code>hospital_server.py</code>	MCP server exposing <code>search_hospitals</code> , <code>nearest_hospital</code> , <code>available_departments</code> .
Servers	<code>emergency_contact_server.py</code>	MCP server exposing <code>notify_family</code> , <code>share_location</code> .
Servers	<code>maps_server.py</code>	MCP server exposing <code>current_location</code> , <code>route</code> , <code>distance</code> .

11. Data Models and Interfaces

Task Intent

```
{
  "task_id": "uuid",
  "task_type": "emergency_orchestration",
  "emergency_type": "cardiac_suspected",
  "patient_name": "John",
  "symptoms": ["chest_pain", "difficulty_breathing", "sweating", "unconscious"],
  "location": "Kakkanad",
  "severity_hint": "high"
}
```

Retrieval Result (hospital candidate)

```
{
  "source_id": "hospital:hospital_a",
  "source_type": "hospital_record",
  "title": "Hospital A",
  "snippet": "Cardiology, ICU, ambulance available, 2.0 km away",
  "semantic_score": 0.88,
  "lexical_score": 0.81,
  "rerank_score": 0.93,
  "metadata": {
    "departments": ["ICU", "Cardiology"],
    "distance_km": 2.0,
    "rating": 4.0,
    "ambulance_available": true
  }
}
```

Context Graph Node

```
{
  "node_id": "hospital:hospital_a",
  "node_type": "Hospital",
  "attributes": {
    "departments": ["ICU", "Cardiology"],
    "distance_km": 2.0,
    "ambulance_available": true
  }
}
```

Execution Plan

```
{
  "plan_id": "uuid",
  "case_id": "uuid",
  "steps": [
    {"order": 1, "action": "call_ambulance", "target": "hospital:hospital_a"},
    {"order": 2, "action": "notify_family", "target": "contact:jane_doe"},
    {"order": 3, "action": "navigate", "target": "hospital:hospital_a"},
    {"order": 4, "action": "share_patient_summary", "target": "hospital:hospital_a"},
    {"order": 5, "action": "connect_physician", "target": "doctor:dr_nair"}
  ],
  "requires_human_approval": true,
  "explanation": "Hospital A selected for Cardiology + ICU availability and shortest distance (2.0 km); ambulance confirmed available."
}
```

12. Core Algorithms

12.1 Context Intelligence Score (adapted)

Combines retrieval relevance with healthcare-specific quality signals (replacing the CIE’s software-

quality signals):

- `semantic_similarity` — symptom/department semantic match
- `proximity_score` — inverse of distance in km
- `department_match` — whether the hospital has a department directly relevant to the symptom cluster
- `resource_availability` — ambulance/bed/doctor availability flag
- `rating` — hospital quality rating (mock data)
- `freshness` — recency of the mock record's last update

```
def context_intelligence_score(item):
    return (
        0.35 * item.semantic_similarity +
        0.25 * item.proximity_score +
        0.20 * item.department_match +
        0.10 * item.resource_availability +
        0.05 * item.rating +
        0.05 * item.freshness
    )
```

12.2 Hybrid Retrieval Pseudocode

```
def hybrid_retrieve(query, k=10):
    lexical = bm25_search(query, top_k=k)           # exact department/symptom terms
    semantic = embedding_search(query, top_k=k)      # symptom -> department meaning
    merged = merge_results(lexical, semantic)
    reranked = cross_encoder_rerank(query, merged)
    return reranked[:k]
```

12.3 Priority Inference (healthcare analog of CIE's architecture-pattern inference)

```
def infer_priority_resources(context_graph):
    priorities = []
    if has_symptom(context_graph, "chest_pain") and has_history(context_graph, "heart_disease"):
        priorities.append("cardiology_priority")
    if has_symptom(context_graph, "unconscious"):
        priorities.append("icu_priority")
    if not has_edge(context_graph, edge_type="ambulance_available"):
        priorities.append("self_transport_flag")
    return priorities
```

12.4 Plan Compression

```
def compress_context(context_graph, priorities, token_budget=800):
    top_hospital = select_top_ranked(context_graph, node_type="Hospital", priorities=priorities)
    top_doctor = select_available(context_graph, node_type="Doctor", hospital=top_hospital)
    contact = select_primary(context_graph, node_type="EmergencyContact")
    return build_compact_context(top_hospital, top_doctor, contact)
```

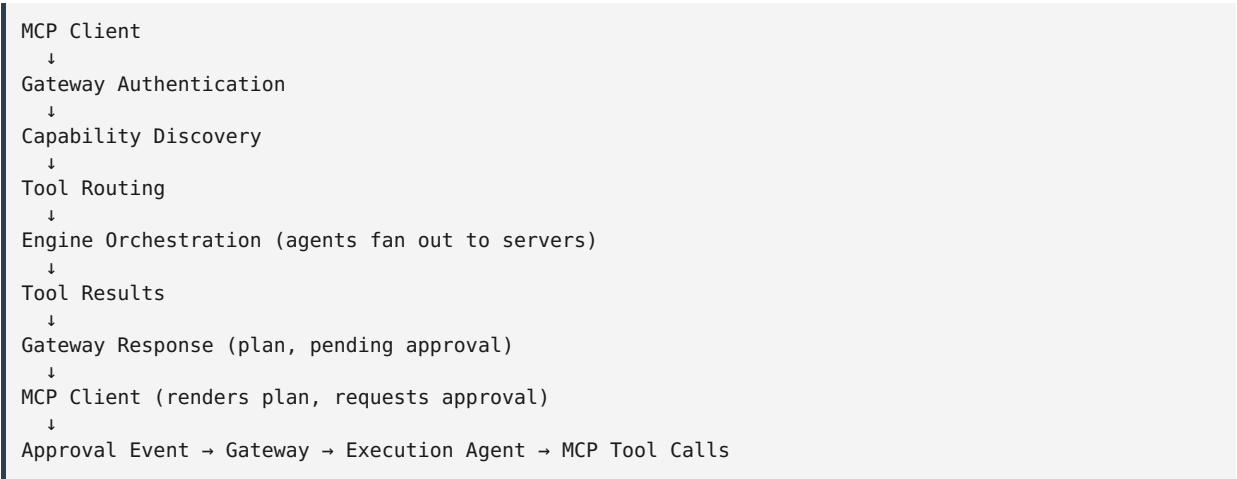
13. Sequence Flows

13.1 End-to-End Request Flow

```
User Text Input
↓
Intent Understanding
↓
Planning
↓
Knowledge Discovery (Patient / Hospital / Doctor / Contact / Maps)
↓
Hybrid Retrieval
↓
Re-ranking
```



13.2 MCP Interaction Flow



14. MCP Tool Specifications

Tool Name	Purpose	Input	Output
get_patient_summary	Fetch patient identity/basic record	patient_name	Patient summary object
get_allergies	Fetch known allergies	patient_id	List of allergies
get_medications	Fetch current medications	patient_id	List of medications
search_hospitals	Find hospitals by department/location	query, location, filters	Ranked hospital list
nearest_hospital	Find closest hospital matching criteria	location, departments	Single hospital record
available_departments	List departments at a hospital	hospital_id	Department list
notify_family	Send notification to emergency contact	contact_id, message	Delivery status
share_location	Share patient/case location with a party	case_id, recipient	Confirmation
current_location	Resolve current location text to coordinates	location_text	Lat/long
route	Compute route between two points	origin, destination	Route/ETA
distance	Compute distance between	origin, destination	Distance in km

	two points		
<code>call_ambulance</code> (<i>execution, approval-gated</i>)	Dispatch ambulance request	<code>case_id, hospital_id</code>	Dispatch confirmation
<code>connect_physician</code> (<i>execution, approval-gated</i>)	Initiate contact with an available doctor	<code>doctor_id, case_id</code>	Connection status

Example MCP-style payload

```
{
  "tool": "search_hospitals",
  "arguments": {
    "query": "chest pain cardiology",
    "location": "Kakkanad",
    "filters": {"ambulance_available": true}
  }
}
```

15. Evaluation Framework

Carried over from the CIE's evaluation approach, retargeted:

Suggested metrics

- Retrieval recall@K for hospital/doctor matches against a labeled mock benchmark.
- Reranker precision@K.
- Priority-inference accuracy (does the system correctly flag cardiology/ICU priority given symptom+history combinations in test cases?).
- Plan completeness rate (does every generated plan address ambulance, hospital, doctor, and contact?).
- Safety-checker catch rate (does `safety_checker.py` reliably reject injected diagnostic/dosing language in adversarial test prompts?).
- Human evaluation of plan explanation coherence.
- End-to-end latency (text input → approved plan).

Benchmark categories (examples/ directory)

- Cardiac emergency
- Trauma emergency
- Respiratory emergency
- Pediatric emergency
- Ambiguous/incomplete input (tests graceful degradation)

16. Roadmap

MVP (hackathon-scale, ~2-5 days)

- Intent parser, minimal planner
- One discovery agent per server (patient, hospital, doctor via hospital server, contact, maps)
- Hybrid retrieval (BM25 + embeddings) and basic reranking
- Context graph (6 node types) and rule-based priority inference
- Plan builder + LLM generation of the ranked plan
- Basic validator (`safety_checker` only)
- Human approval gate (simple CLI/UI confirmation)

v0.3 (this blueprint, ~4-8 weeks)

- Full MCP gateway (auth, session, cache, logging, capability discovery)
- All engine layers and specialized agents
- Full validation pipeline (schema + safety + completeness + explanation)

- Evaluation suite with labeled mock benchmarks
- Repository/documentation scaffolding for open-source contributors

v1

- Session-aware multi-turn case handling (a case evolving over time, not single-shot)
- Ranking signals beyond similarity (real hospital load/capacity signals, if partnered data becomes available)
- Feedback loop: approved/rejected plans improve `context_intelligence_score` weighting
- Optional real-data connectors behind the same MCP server interfaces (opt-in, clearly marked non-production)

17. Research Differentiation

The strongest claim, carried directly from both source blueprints and unified here:

“A Context Intelligence Engine that uses MCP, hybrid retrieval, reranking, and agentic planning to orchestrate healthcare resources based on context — not diagnosis — while preserving patient privacy.”

The differentiating shift, in the CIE’s own framing, generalizes cleanly to healthcare:

- **From resource retrieval to context-aware orchestration.** Instead of “here are five nearby hospitals,” HCIE produces: *“Two Cardiology-capable hospitals were found within 3 km. Hospital A has ambulance and ICU availability; Hospital B does not. Plan will route to Hospital A and notify the registered emergency contact.”*
- This is the same “file retrieval → architecture retrieval” leap the general CIE claims for code, applied to “hospital listing → context-aware care routing.”
- Because the underlying engine (intent → planning → discovery → hybrid retrieval → context graph → plan generation → validation) is domain-agnostic, HCIE also serves as a **proof-of-concept that the general CIE architecture generalizes beyond software engineering** — a meaningful research contribution in its own right.

18. Open-Source Strategy

Recommended public artifacts

- `README.md` — vision, demo GIF/video, architecture summary, explicit non-diagnostic disclaimer.
- `docs/prd/` — product requirements (this document serves as the seed).
- `docs/research/` — literature notes on MCP orchestration, hybrid retrieval/reranking in healthcare IR, and context-graph reasoning.
- `docs/architecture/` — diagrams for gateway, engine layers, agent tree, context graph schema.
- `examples/` — the benchmark case files listed in Section 15.
- `tests/evaluation/` — reproducible comparisons against a plain-RAG baseline (retrieve-and-stuff, no graph, no reranking) to demonstrate the engine’s value-add.
- A clearly labeled `DISCLAIMER.md`: all patient/hospital/doctor data is mock; the system does not diagnose; execution tools are approval-gated and disabled by default outside the demo mock servers.

Initial milestones

1. Publish this unified blueprint.
2. Implement the MVP slice (Section 16).
3. Add hybrid retrieval + reranking.
4. Add context graph + priority inference.
5. Benchmark against a plain-RAG baseline.

6. Open contribution guidelines, issue templates, and a “good first issue” set targeting individual discovery agents or MCP tools (naturally parallelizable across contributors).

End of blueprint.